

A Spreadsheet Engine, Seven Layers Deep (EN)

From LeetCode 631 to industrial design: seven layers of one problem

Compiled 2026-07-27 · companion code: fuse2.cpp

How to use this document: read once end-to-end (~35 min). Chapter 9 (case history) gets one more pass on interview morning — every entry is a bug you personally hit, one glance re-activates it. One governing rule: **in the room you hand-write V1 only (Ch. 3-5); everything from V2 onward is delivered verbally, and coded only if the interviewer explicitly asks.**

1. Intelligence: source, rubric, success bar

1.1 Why this problem

All three published Fuse interview reports (2025 Q2, 2026 Q1, 2026 Q2) feature the **same problem**: a simplified Excel engine (SET / GET / SUM + cycle detection, UNDO as follow-up). The official prep doc confirms the genre: "a lightweight in-memory system (often similar to a spreadsheet or table-like model)." The base is LeetCode 631 (official Hard); Fuse adds cycle detection + transactional semantics on top.

Tail risk: the prep doc says the problem "may vary." Hedge: 90% of effort on this problem, 10% on the transferable process — clarify spec → minimal model → pure read path → transactional write path → trace → state trade-offs. The problem may change; the process does not.

1.2 Official rubric mapped to concrete moves

Rubric line	What it means here
problem-solving ability	correct graph model, the cycle invariant, transaction ordering
clean & maintainable code	Coordinate / Cell / Graph layering, ForEach extraction, pure Get
extending an existing solution	each new requirement = "which few lines change", never a rewrite
clarity of reasoning & communication	one-line plan before typing; trace aloud after; speak decisions ("duplicates count per occurrence — flagging it")
appropriate data structures	hash maps, adjacency sets, compressed ranges
correctness as data changes	invalidation propagation; "a cache without invalidation is a bug"

1.3 The three top-candidate bars (prep-doc wording)

- **"finish with 10-15 mins to spare"** — V1 $\approx$ 150 lines incl. blanks, budget 40-45 min, leaving 15 for discussion layers;
- **"clearly explain R/W patterns"** — write-heavy $\rightarrow$ lazy; read-heavy $\rightarrow$ cache-with-invalidation; ultimate $\rightarrow$ delta propagation;
- **"discuss multithreading / partial caching"** — three-tier answer, Ch. 8.

Interviewer: Dmitrii Zyrianov, 10+ yrs enterprise backend. His lens: interfaces, error handling, extensibility — not speed-solving. Platform: codeshare, **no execution** — your test runner is tracing aloud.

2. Full spec and the three buried mines

2.1 Spec

```
Set(cell, value)      set cell to an integer literal (overwrite semantics)
Sum(destCell, refs)   set destCell to a SUM formula (overwrite semantics). refs elements:
                      single cell "B2", or rectangle "A1:B2" (inclusive)
Get(cell)             current value; formulas evaluate live; empty cells  $\rightarrow$  0
```

Example:

```
Set("A1",1); Set("B2",2)
Sum("D1", ["A1:B2","B2"]) # D1 = A1+A2+B1+B2 + B2 = 5
Get("D1") # 5
Set("A1",2); Get("D1") # 6
```

Cycles: a Sum that would create a (direct or transitive) cycle MUST raise an error AND leave the sheet unchanged.

```
Sum("A1",["A1"])  $\rightarrow$  error; Sum("A1",["B1"]); Sum("B1",["A1"])  $\rightarrow$  2nd errors
```

Scale: Excel-style coords (multi-letter columns like AA10; positive rows),
~1e5 ops, sparse sheet. Follow-up: UNDO (Set and Sum both count as writes).

2.2 The three mines ("You WILL miss the most critical requirement")

#	Mine	Defuse move (say during clarification)
1	Duplicates count per occurrence: B2 appears in the range AND as a single ref $\rightarrow$ added twice ($5 = 3 + 2$). Most likely "the" mine.	Point at the example: "B2 counted twice, correct?"
2	Multi-letter columns: bijective base-26, A=1 ... Z=26, AA=27. Must be <code>col*26 + (c-'A'+1)</code> ; forget the +1 and A collides with AA at 0.	Say "+1, bijective base-26" while writing Coordinate
3	Sparse sheet, never materialize empties: no <code>operator[]</code> on read paths; big ranges are iterated, never instantiated.	Mention "avoiding materializing empty cells" when writing Exist

Also clarify: empty Get returns 0 (vs throw); Sum is **overwrite** (two Sums → latter wins; Set over Sum and vice versa likewise).

3. Layer 1 · Data model: a cell's identity

At any moment a cell has exactly one identity: literal XOR formula. Writes overwrite the whole state.

Corollary 1: a formula's value lives **nowhere** — computed on Get (V1) or held by a cache *with invalidation* (V2).

Corollary 2: a cell is never an "operation list" — no accumulate-ops-and-replay semantics (this wrong model cost three rework rounds; case #1).

Corollary 3: future operators (+, −, ×) change the *inside* of "formula" (an expression tree), not the cell's shape.

```
struct Cell {
    int val = 0;                // literal's value; formulas: 0 (V1) or cached (V2)
    bool valid = false;         // V1: true=literal, false=formula (type tag)
                                // V2: doubles as cache-freshness for formulas (Ch.6)
    vector<vector<Coordinate>> depends; // size==1 -> single ref; size==2 ->
    [topLeft,bottomRight]
    Cell() {}
    Cell(int v) { val = v; valid = true; }
};
```

Notes: the `= 0` default is load-bearing — an uninitialized `val` once got default-constructed via `operator[]` and summed garbage (UB; case #3). Ranges are stored compressed (two corners), the first brick of Ch. 7's memory strategy. Optional line for the room: "in production I'd make the exclusivity explicit with `std::variant<int, Formula>`; the bool keeps it simple here."

4. Layer 2 · Parsing and evaluation: a pure GetHelper

```
struct Coordinate {
    int row, col; string name;                // name "row:col" = graph key (debuggable)
    Coordinate(const string & cell) {         // "AA10" -> col=27, row=10
        size_t i=0; row=0; col=0;
        while (i<cell.size()) {
            if ('A'<=cell[i] && cell[i]<='Z') col = col*26 + (cell[i]-'A'+1); // mine #2
            else if ('0'<=cell[i] && cell[i]<='9') row = row*10 + cell[i]-'0';
            i++;
        }
        name = to_string(row) + ":" + to_string(col);
    }
    Coordinate(int r,int c){ row=r; col=c; name = to_string(r)+":"+to_string(c); }
};
```

Notes: the parser silently skips unknown chars — convenient (colon passes) but permissive ("a1", "1A"); one sentence covers it: "in production I'd validate and throw." name is "row:col", not "AA10": avoids the ~10-line reverse base-26 conversion for range-expanded cells. Forgetting name in the (int,int) ctor once made all range cells collide as "" in the graph (case #7).

```

template <typename Func>
void ForEachDependencyCell(const vector<Coordinate> & dep, Func && f) {
    if (dep.size()==1) { f(dep[0]); return; }
    for (int r=dep[0].row; r<=dep[1].row; r++)
        for (int c=dep[0].col; c<=dep[1].col; c++) f(Coordinate(r,c));
}

```

Notes: extracted at the *third* occurrence of the single-or-rectangle skeleton — timing matters: first-pass extraction is premature abstraction; third-repeat extraction, announced aloud, is refactoring instinct. Callers needing early exit (cycle check) use an external flag + return.

```

bool Exist(const Coordinate & co) {           // sparse gatekeeper: query, never insert (mine #3)
    if (!_cache.count(co.row)) return false;
    if (!_cache[co.row].count(co.col)) return false;
    return true;
}
int GetHelper(const Coordinate & t) {         // *** PURE in V1: returns, never writes ***
    if (!Exist(t)) return 0;                 // empty -> 0 (clarified default)
    Cell & cell = _cache[t.row][t.col];
    if (cell.valid) return cell.val;         // literal: 0(1)
    int val = 0;
    for (const auto & dep : cell.depends)
        ForEachDependencyCell(dep, [&](const Coordinate & co){
            if (Exist(co)) val += GetHelper(co); // duplicates add naturally (mine #1)
        });
    return val;
}

```

The rule paid for with three incidents: in V1, `_cache` never appears on the left of an assignment inside the Get family. Writing results back = a cache without invalidation = a bug: doubled second reads (5→10), pointless literal write-backs, and one typo that destroyed user data on a read path. A real cache is V2's separate project.

Complexity line: Set $O(1)$; Get $O(\text{dependency tree})$ — lazy's price sheet: writes cheap, reads pay. Write-heavy: this is the final answer. Read-heavy: enter V2. Say this trade-off *before* writing Sum (rubric: R/W patterns).

5. Layer 3 · Cycle detection and transactions

```

// EDGE DIRECTION: dep -> target (a cell points at the formulas that USE it)
// i.e. "who is affected when I change". Serves BOTH cycle check (here)
// and invalidation (Ch.6) -- one graph, two duties.
struct Graph {
    unordered_map<string, unordered_set<string>> data;
    void addEdges(const string& f, const string& t){ data[f].insert(t); }
    void removeEdges(const string& f, const string& t){
        if (!data.count(f)) return;
        data[f].erase(t); // once wrote data.erase(t): nuked t's whole list (case #9)
    }
    // Would adding from->to create a cycle?
    // INVARIANT: graph acyclic before every Sum => any new cycle contains the
    // new edge => ONE question: can we walk from `to` back to `from`? One DFS.
    bool AddEdgeHasCycle(const string& from, const string& to){

```

```

    if (from == to) return true;          // self-ref: no edges exist yet; MUST
                                         // special-case (case #11; named test)
    unordered_set<string> visited;        // visited != cycle! diamonds revisit legally
    visited.insert(to);
    function<bool(const string&)> dfs = [&](const string& r)->bool{
        if (!data.count(r)) return false;
        for (auto& n : data[r]) {
            if (n == from) return true; // back at origin -> cycle
            if (!visited.count(n)) { visited.insert(n); if (dfs(n)) return true; }
        }
        return false;
    };
    return dfs(to);
}
};

```

Why not topological sort: toposort certifies the *whole* graph, $O(V+E)$ + in-degree bookkeeping; the invariant reduces the question to "does the new edge close a loop", answered by one DFS over the reachable subgraph in ~ 12 lines with zero extra state. Same verdict, strictly cheaper — the minimal-tool principle. Also: "visited = cycle" is a false test (case #2): in diamond $D1 \rightarrow \{B1, C1\} \rightarrow A1$, $A1$ is visited twice, no cycle, and its value legitimately counts twice. The only cycle criterion is returning to the origin.

5.1 Sum's transaction (your chosen order: delete-first, restore-on-failure)

```

void Sum(const string& target, vector<string>& cells){
    Coordinate co(target);
    // 1. BACKUP (for restore)
    Cell old; bool had = false;
    if (Exist(co)) { had = true; old = _cache[co.row][co.col];
                    RemoveEdgeHelper(co); } // drop target's OWN old out-edges
    // 2. PARSE refs -> new formula (compressed ranges)
    Cell nc;
    for (auto& s : cells){
        size_t p = s.find(':');
        vector<Coordinate> dep;
        if (p==string::npos) dep.push_back(Coordinate(s));
        else { dep.push_back(Coordinate(s.substr(0,p)));
                dep.push_back(Coordinate(s.substr(p+1))); } // p+1 skips ':'
        nc.depends.push_back(dep);
    }
    _cache[co.row][co.col] = nc; // 3. COMMIT candidate
    // 4. CHECK+LINK edge by edge (helper stops on first cycle and rolls back
    // the edges it just added)
    bool cyc = false; addEdgeHelper(co, cyc);
    // 5. FAILURE: restore pre-call state + raise
    if (cyc){
        if (had){ _cache[co.row][co.col] = old; // 5a restore + rebuild old edges
                  bool d=false; addEdgeHelper(co, d); }
        else { _cache[co.row].erase(co.col); } // 5b cell never existed: erase
                                                  // (missing this once left a
                                                  // poisoned formula; case #12)
        throw runtime_error("cycle detected"); // spec REQUIRES an error
    }
}

```

Two legitimate orderings: (a) check-first: old edges are target's own out-edges, and DFS terminates on reaching target, so they can't affect the verdict — zero rollback needed; (b) delete-

first (current): the failure branch must restore. Verdicts are equivalent; either passes — **what's graded is that the failure branch closes and throws**. Free-points move: ask beforehand "on cycle, should the sheet remain unchanged, or is fail-fast acceptable?"

5.2 V1 deliverable (everything you hand-write Tuesday)

Coordinate 25 + Cell 15 + Exist/Get/ForEach 30 + Graph 30 + Set/Get/Sum 35 + main asserts 15 ≈ 150 lines, 40–45 min. **Fixed assert set:** official example (5→6); `Sum("B3",["B3"])` throws; two-step cycle A1→B1→A1 second call throws AND Get still works after; one overwrite case each way (Set over formula, Sum over literal).

6. Layer 4 (V2) · Cache and invalidation

Verbal-delivery layer. Code lives in fuse2.cpp, compiled, asserts green.

Lazy re-scans on repeated Gets → formula cells get a cached value. **The soul of a cache is invalidation:** after Set/Sum, walk "who is affected by me" and mark downstream formulas dirty — and that direction is exactly the dep→target graph the cycle check already built. **One graph, two duties.** Fresh cache hit: O(1); dirty: recompute and refill.

```
// V2 GetHelper tail:
cell.val = val; cell.valid = true; // the SAME two lines that were three incidents
return val;                       // in V1 are correct DESIGN here -- the sole
                                  // difference is that invalidation now exists.

void InvalidateDownstream(const string& src){ // BFS along dep->target
    unordered_set<string> vis; queue<string> q;
    vis.insert(src); q.push(src);
    while(!q.empty()){
        string cur=q.front(); q.pop();
        if (!_graph.data.count(cur)) continue;
        for (const auto& nxt : _graph.data[cur]){
            if (!vis.insert(nxt).second) continue; // diamond-safe
            q.push(nxt);
            if (_sum_ranges.count(nxt)) { ... mark that formula cell dirty ... }
        }
    }
}
// called after Set commits, and after a successful Sum (marking itself dirty too)
```

Notes: cleaner design = separate `bool dirty` (type tag vs state tag decoupled) — mention it; reusing valid is self-consistent under "literals are always valid". Propagation is transitive (whole downstream closure in one BFS) and mark-only — recomputation is deferred to the next Get. Eager recompute needs topological order; contrast verbally. Ultimate: **delta propagation** — on Set push (new–old)×multiplicity to dependents, no recomputation at all; saying it locks the strong signal.

6.1 The final big bug: coverage edges

If edge-building requires Exist(cell), "referenced first, materialized later" cells lose their edges. Two failure modes: (1) missed cycle → infinite recursion: Sum("D1",["A1"]) (A1 absent, no edge) then Sum("A1",["D1"]) passes the check, a real cycle commits; (2) silent staleness: Sum("F1",["F2"]) then Sum("F2",["A1"]) — edge F2→F1 missing, later Set("A1") never invalidates F1, Get("F1") returns a stale cache. **Root cause in one line: graph existence ≠ value existence.** Dependencies are declared by formulas, regardless of whether the cell currently holds a value.

Fix: every operation that *materializes* a cell — Set AND Sum — runs AttachCoverageEdges: scan each formula's registered ranges (`_sum_ranges`), if the new cell falls inside, add edge cell→formula (Sum's path also feeds this into the same cycle check; failures roll the attached edges back). Two regression asserts reproduce both scenarios; compiled and green. The full loop — diagnosis (yours) → patch (delegated) → regression asserts → machine verification — is itself the answer if testing comes up.

7. Layer 5 · Ranges and the "absent cell" trade

Tier	Approach	Cost	Verdict
Full expansion	range → per-cell edges	A1:ZZ9999 → millions of edges	never
Compressed (current)	store corners; point-in-rectangle checks	O(ranges per formula) per query	correct at 1e5 ops
Interval index	interval tree: point → covering ranges	O(log n + k), heavier code	only if ranges explode; "I'd need real contention data first"

Absent-cell option	Mechanism	Pays	Buys
Placeholder keys	edges added regardless of existence (keys are just strings)	a few hash keys per formula	O(1) lookup on Set
Scan formula ranges (current)	at materialization, scan <code>_sum_ranges</code> for coverage	O(formulas × ranges) per materialization	zero placeholder memory; cheap when formulas are sparse

Decision rule in one line: **Set frequency vs formula density.** (Second use of the R/W-patterns rubric line.)

8. Layer 6 · Undo / Layer 7 · Concurrency (verbal)

8.1 Undo

- **vs transaction rollback:** Sum's internal backup is failure-rollback, discarded on return; Undo reverses the last *successful* write — history must persist. Similar machinery, different lifetime.
- **Design:** history stack of (coord, full old Cell state, existed?) . **Full state, not just the value** — the old state may be a formula; storing an int degrades it to a constant (the published reference solution has exactly this bug; pointing it out is an overtake). Undo = pop → restore whole cell (or erase) → graph sync (reuse Remove/add/AttachCoverage helpers) → invalidate downstream.
- **Clarify:** failed Sums don't enter the stack (nothing changed); empty stack: no-op or throw (pick and flag). Redo: two stacks; any new write clears the redo stack.

8.2 Concurrency, three tiers

Anchor sentence: lock granularity should follow operation scope — Get reads a dependency tree, Sum's check reads the whole graph, so operations are graph-scoped and I start with one reader-writer lock.

1. **shared_mutex baseline:** Get shared, Set/Sum exclusive; check-and-commit is atomic for free. **V2 caveat:** Get refills the cache = a write under a shared lock — atomics / a small cache lock / lock upgrade; naming this layer under questioning is real credit.
2. **Read-dominant:** snapshot / copy-on-write — writers build a new version and swap; readers lock-free on the old.
3. **Fine-grained:** region shards + ordered lock acquisition; "an order of magnitude more complexity — I'd need contention data first."

Why not cell-level locks: semantics, not just rehash mechanics — operations span cells; locking one cell can't stop "you changed A1 while I was summing D1". **Why not lock-free:** CAS fits fixed small critical sections; the write scope here is a dynamic subgraph. "I've built lock-free structures; wrong tool here" — showing the arsenal and the judgment not to draw it.

9. Case history: every pit, five days

#	Pit	Symptom	Lesson
1	cell = op list	three rounds of Sum-semantics rework	cell is identity (literal XOR formula), not a log; pre-built op frameworks = premature abstraction
2	"visited = cycle"	diamonds misjudged	only returning to origin is a cycle
3	uninitialized val	garbage summed via operator[]	default-init members; UB is what senior eyes catch first
4	cache w/o invalidation (×3)	doubled reads; pointless write-backs; a read destroying data	V1 rule: no _cache on the left inside Get
5	operator[] on read path	a big range materialized millions of empties	count/find on reads; [] on writes only
6	base-26 without +1	A collides with AA	bijective: A=1; say it aloud while writing
7	(int,int) ctor missing name	range cells all named "" in the graph	both ctors keep the invariant — or key by coordinates
8	for (auto dep : ...)	vector deep-copied per iteration	const auto&; const string& on signatures
9	data.erase(to)	wiped the other node's whole adjacency list	data[from].erase(to); think container level first
10	spoken direction ≠ coded direction (×2)	"did I build it backwards?" spiral	both mirrors are correct; the discipline is consistency — type the direction comment before code
11	self-reference missed	Sum("B3",["B3"]) → stack overflow on Get	from==to special case on top; a named test case
12	new-cell cycle not cleaned	poisoned formula lingers → infinite recursion	failure cleanup = restore old AND erase new; half is none
13	coverage only in Set	missed cycle / silent staleness	graph existence ≠ value existence; every materialization point attaches coverage
14	throw kept slipping	silent failure, caller blind	"raise an error" is an acceptance criterion, not style
15	tests skipped twice (Harvey mock)	"easy, saving time" / feature with zero asserts	a feature without a test didn't happen; assert reflex after every function

9.1 Three meta-patterns (worth more than any single pit)

1. **Write first, debate second:** the naive-correct version lands first; optimizations get announced, not written. The cure for "not thought through yet" is not more thinking — writing IS the thinking organ; the felt moment of "now it's clear" is produced by writing, never precedes it.
2. **Write only what's asked:** every field and every if must point at a requirement; what can't point is a bet. Itchy optimizations go into a spoken parking lot (line 10.3).
3. **Trace aloud after every block:** in a no-execution editor, tracing is the test runner. Typos (= val vs = sub_val) and copy-paste slips (dep[0] vs co) die there and nowhere else.

10. Talk library (中文意图版 —— 理解要点,英文自己长出来)

每一段给的是"这一刻要传达什么"+ 两三个英文锚点词。不背句子,背意图;锚点词保证关键术语不临场卡壳。

10.1 开场澄清(前 3-5 分钟)

要传达的:动笔前把五个语义钉死——①空格 Get 返回 0 吗;②样例里 B2 出现两次,按次数累加对吗(指着例子问);③Sum 是覆盖语义(等于赋值)对吗;④列可以多字母(AA10)吗;⑤检出环时,表格必须保持不变,还是 fail-fast 可接受?

锚点: counted twice / per occurrence · overwrite / assignment semantics · remain unchanged vs fail-fast

10.2 方案陈述与读写权衡(写码前 1 分钟)

要传达的:我先写"直白的 lazy 版"拿正确性:cell 要么字面量要么公式,Get 递归现算且是纯函数;Set O(1)、Get 付依赖树的钱——写多读少这就是终态。如果读占主导,我已经看到两条升级路(dirty 缓存 + 沿依赖图失效;delta 传播),等基础版跑对了随时可去。

核心姿态:naive 先行是选择,不是水平;优化是预告片,不是正片。

锚点: straightforward lazy version for correctness · pure function · write-heavy vs read-heavy · dirty-flag caching · delta propagation

10.3 泄压阀(手痒想优化时)

要传达的:"这里可以加缓存/delta——我先记下,性能成为需求时再回来。"把优化欲望说出来存档,代码保持干净。这句是你"想到就想写"冲动的唯一出口。

锚点: I'll note it and come back if performance becomes a requirement

10.4 判环方案(写 Graph 前)

要传达的四层,顺序说:①边的方向:格子指向依赖它的公式("谁受我影响")——将来失效传播用同一方向,一图两用;②不变量:每次 Sum 前图无环,所以新环必过新边;③于是只问一件事:从 target 沿现有边能否走到某个新 ref,一次 DFS 解决,不需要拓扑排序;④被问"每次 Sum 都 DFS 贵吗":检查必须在落盘前做,DFS 只走可达子图,这是正确性保证的最低成本。

锚点: edges point at dependents · one graph serves both · invariant: acyclic before each Sum · any new cycle passes through the new edge · one DFS, reachable subgraph

10.5 重构时机(同一骨架第三次出现时)

要传达的:"我注意到 range 展开逻辑在重复了——抽一个 helper。"当着面试官的面完成小重构,是 refactoring instinct 的现场证据;注意必须是**第三次重复之后**,第一遍就抽 = 过早抽象 + 背题嫌疑。

锚点: I notice I'm repeating ... let me extract

10.6 缓存与失效(被追问 repeated gets 时)

要传达的:公式格加缓存值 + dirty 位;缓存的灵魂是失效——Set 后沿依赖图 BFS 把下游全标脏;没有失效机制的缓存不是优化,是 bug(这句你有三次亲测,说出来有质感)。终极版 delta 传播:Set 时把 (新值-旧值) × 引用重数 直接推给依赖者,连重算都省。

锚点: dirty bit · the soul of the cache is invalidation · a cache without invalidation is a bug · delta propagation · multiplicity

10.7 未存在格子与 range(被追问内存/#4 时)

要传达的:①原则:图存在性 ≠ 值存在性——依赖由公式声明,与格子当下有没有值无关;②两档付法:占位键入图 (Set O(1),付每条公式几个哈希键的内存)vs 物化时扫公式区间表(零占位内存,付 O(公式数) 查询)——选哪档看 Set 频率 vs 公式密度;③range 永不展开:存两个角,点 ∈ 矩形判断;range 极多才升 interval tree,1e5 规模线性扫就够,不提前上。

锚点: graph existence isn't value existence · placeholder keys vs scanning registered ranges · point-in-rectangle · interval tree, but not prematurely

10.8 并发(收官)

要传达的:①定调:锁粒度跟随操作作用域——Get 读整棵依赖树、判环读全图,操作是图作用域的,所以从一把读写锁开始(Get 共享、写独占,检环+落盘天然原子);②读极多升级 copy-on-write 快照;③我写过 lock-free,但它适合固定小临界区,这里写作用域是动态子图——不匹配,不用。亮兵器 + 不乱拔刀。

锚点: lock granularity follows operation scope · reader-writer lock · check-and-commit atomic for free · copy-on-write snapshot · lock-free fits fixed small critical sections

10.9 经验收官句(聊 testing / 设计哲学时)

要传达的:我在这题上的规矩:先把关系做对(建边、回滚、失效),再谈性能;每写完一个功能立刻补 assert——在不能运行代码的编辑器里,出声 trace 样例就是我的测试运行器。

锚点: relationships right first, performance second · tracing aloud is my test runner

11. The 60-minute script and final checklist

Time	Action	Output
0-5'	Clarify (10.1) + plan (10.2) + type the direction comment	spec aligned; interviewer's nod = start signal (not inner perfection)
5-20'	Stages 1+2: Coordinate → Cell → Exist → GetHelper → Set/Sum (no cycles yet)	trace the official example aloud: 5 → 6
20-35'	Stage 3: Graph + AddEdgeHasCycle (self-ref!) + transactional Sum + throw	trace self-ref & two-step cycle; four assert groups in main
35-45'	Polish: const refs; ForEach extraction (only if time; else line 10.5)	"core solution is solid"
45-58'	Discussion layers, released per question: undo → cache/invalidation → delta → absent-cell trade → range tiers → concurrency tiers	top-candidate signal
58-60'	Your questions (prepare two: an engineering-challenge one about their telemetry/trading stack; "does this engine have a shadow in one of your real systems?")	engagement signal

11.1 Four mechanical rules, all 60 minutes

1. API signatures and structs first; every field must answer to a requirement line;
2. anything that can't point at a requirement is a bet — don't write it;
3. on each new requirement, first say "this changes function X at place Y", then type;
4. itchy optimizations go through line 10.3, never into code.

11.2 Morning-of checklist

- base-26 +1; name generated in both ctors; no `operator[]` on read paths; `const string&`;
- no `_cache` on the left inside the Get family (V1);
- Sum order: backup → drop old edges → commit → check+link → (fail: restore/erase + **throw**);
- from==to special case; visited ≠ cycle;
- B2 twice; empty = 0; Sum overwrites;
- after every block: trace aloud; after every feature: assert in main;

- autocomplete off was rehearsed; 6pm GMT+1 = 1pm ET, join 15 min early;
- anchor: the first pass is a thought-through draft, not a finished product; slow-and-steady beats fast-and-broken.

One-line summary: V1 is hand skill (45 minutes, four assert groups, tracing aloud); everything after V1 is verbal ammunition — and every sentence of it stands on a pit you personally climbed out of. In that room, you are the person who knows this problem best.